Table of Contents

Abstract 2
Introduction 3
Problem statement 5
Methodology 6
System Requirements 10
System Architecture 12
Flowchart 15
Results and Output Analysis 19
Limitations 26
Future Scope 27
Conclusion 28
References 29

Abstract

Neural Style Transfer is a deep learning technique that combines the structural content of one image with the artistic style of another image to generate a visually appealing stylized output. This project implements an optimization-based Neural Style Transfer system using a pretrained Convolutional Neural Network (CNN), specifically the VGG19 model. The pretrained network is utilized as a fixed feature extractor to obtain hierarchical representations of both content and style images.

The structural information of the content image is preserved using content loss computed from deep feature maps, while the artistic characteristics of the style image are captured using Gram matrices that represent feature correlations. A combined loss function, consisting of weighted content and style losses, is minimized using the Adam optimization algorithm. During this process, the parameters of the pretrained network remain frozen, and only the output image is iteratively optimized.

The system supports multiple artistic styles, including Starry Night, Watercolor, Oil Painting, Abstract Art, Psychedelic Swirl Art, and Color Pencil Art. Experimental results demonstrate that the proposed implementation effectively preserves content structure while transferring stylistic textures and patterns. The project highlights the application of transfer learning, feature extraction, loss-based optimization, and deep convolutional architectures in artistic image generation.

Introduction

With the rapid advancement of Artificial Intelligence and Machine Learning, deep learning techniques have significantly improved the field of computer vision. Convolutional Neural Networks (CNNs), in particular, have demonstrated remarkable performance in tasks such as image classification, object detection, and image segmentation. Beyond analytical tasks, deep learning has also enabled creative applications such as artistic image generation through Neural Style Transfer.

Neural Style Transfer (NST) is a deep learning technique that synthesizes a new image by combining the structural content of one image with the artistic style of another image. The method was first introduced by Gatys et al. (2015), who demonstrated that the feature representations learned by deep convolutional networks can effectively separate content and style information. This discovery opened new possibilities in computational creativity and digital art.

Figure : Concept of Neural Style Transfer combining content and style images

Content Image **Style Image** **Stylized Output**

The fundamental idea behind Neural Style Transfer is that different layers of a pretrained CNN capture different levels of image abstraction. Early layers detect low-level features such as edges and textures, while deeper layers capture high-level semantic structures such as object shapes. By leveraging these hierarchical feature representations, it becomes possible to preserve the structure of a content image while transferring the texture patterns of a style image.

In this project, a pretrained VGG19 Convolutional Neural Network is used as a fixed feature extractor. Content loss is computed to preserve structural information, while style loss is calculated using Gram matrices to capture texture correlations between feature maps. A

combined loss function is minimized through iterative optimization using the Adam optimizer. The output image is gradually refined until it satisfies both structural and stylistic constraints.

The developed system supports multiple artistic styles including Starry Night, Watercolor Painting, Oil Painting, Abstract Art, Psychedelic Swirl Art, and Color Pencil Art. The objective of this project is to demonstrate the practical implementation of Neural Style Transfer using transfer learning and deep feature extraction techniques within the domain of Machine Learning and Pattern Recognition.

Problem statement

The objective of this project is to design and implement a deep learning-based system capable of generating artistic images by combining the structural content of one image with the stylistic features of another image. Traditional image processing techniques are limited in their ability to capture high-level artistic patterns such as brush strokes, textures, and abstract visual representations. Therefore, there is a need for a method that can effectively separate and recombine content and style information using learned feature representations.

The problem addressed in this project is to develop an optimization-based Neural Style Transfer framework that:

1. Preserves the structural layout and semantic content of a given content image.
2. Extracts and represents the artistic style of a selected style image.
3. Generates a new image that integrates both content and style in a visually coherent manner.

To achieve this, the system utilizes a pretrained Convolutional Neural Network (VGG19) as a fixed feature extractor. The challenge lies in appropriately defining and balancing content loss and style loss such that the generated image maintains recognizable structure while exhibiting distinct artistic texture. The model must iteratively optimize the output image to minimize the combined loss function without retraining the neural network.

The final system is required to support multiple artistic styles and produce high-quality stylized outputs for diverse input images, including landscapes, animals, and human portraits.

Methodology

This project implements an optimization-based Neural Style Transfer framework using a pretrained Convolutional Neural Network (CNN). The methodology consists of feature extraction using VGG19, representation of artistic style using Gram matrices, formulation of loss functions, and iterative optimization of the output image.

1. Overall Approach

The proposed system follows these steps:

1. Load the content image and style image.
2. Use a pretrained VGG19 network as a fixed feature extractor.
3. Extract feature maps from selected convolutional layers.
4. Represent style using Gram matrices.
5. Define content loss and style loss.
6. Combine losses into a total loss function.
7. Optimize the output image iteratively using the Adam optimizer.

The parameters of the pretrained CNN remain frozen throughout the process. Only the output image is updated during optimization.

2. Convolutional Neural Network (VGG19)

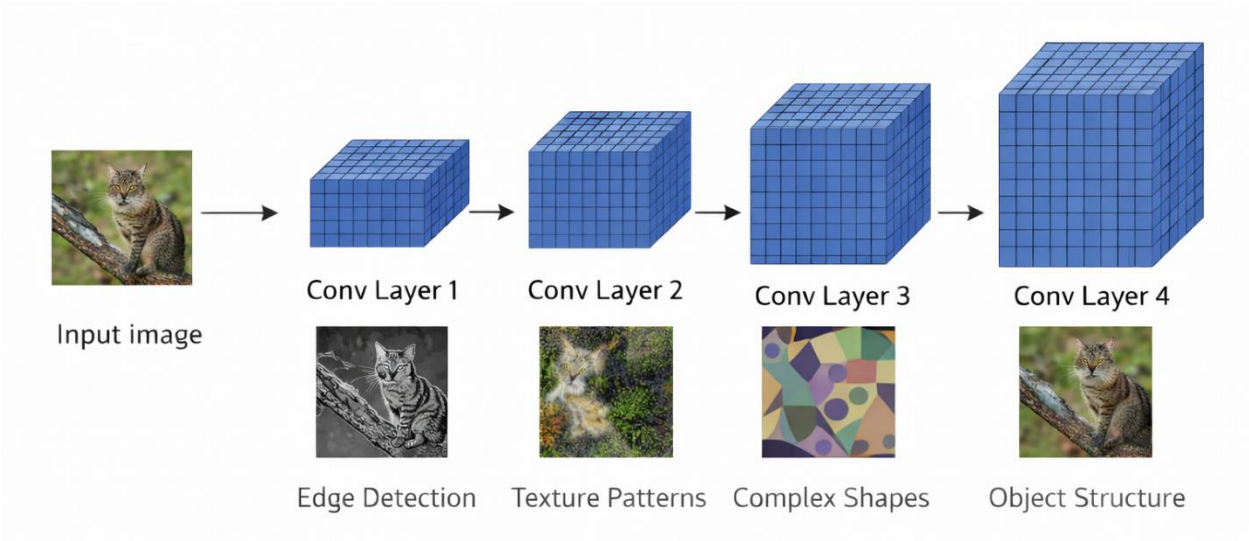
A pretrained VGG19 model is used for hierarchical feature extraction. VGG19 is a deep Convolutional Neural Network trained on the ImageNet dataset. It consists of multiple convolutional layers followed by activation functions and pooling layers.

In this implementation:

- The network is used only for feature extraction.
- Fully connected classification layers are not used.
- Selected convolutional layers are used to extract feature representations.

Early layers capture low-level features such as edges and textures, while deeper layers capture high-level structural information.

Figure : Hierarchical feature extraction in a CNN.



3. Feature Extraction

The content and style images are passed through the VGG19 network. Feature maps are extracted from specific convolutional layers.

Let:

F_l(I) represent the feature map of image I at layer l.

Content features are extracted from deeper layers to preserve structural information.

Style features are extracted from multiple layers to capture texture and patterns.

These feature maps are used to compute content and style representations.

4. Style Representation using Gram Matrix

Artistic style is represented using a Gram matrix.

For a feature map F of size (C × H × W):

1. The feature map is reshaped into a matrix of size (C × HW).
2. The Gram matrix G is computed as:

$G = F \times F^T$

The Gram matrix captures correlations between feature channels.
It represents texture patterns independent of spatial location.

This allows the system to transfer artistic textures while preserving the structure of the content image.

5. Loss Functions

The model uses two primary loss components.

5.1 Content Loss

Content loss measures the difference between feature maps of the output image and the content image.

It is defined as:

$L_content = ||F_l(Output) - F_l(Content)||^2$

This ensures that the structural layout of the content image is preserved.

5.2 Style Loss

Style loss measures the difference between Gram matrices of the output image and the style image.

It is defined as:

$L_style = ||G_l(Output) - G_l(Style)||^2$

This ensures that the texture and artistic patterns of the style image are transferred to the output image.

5.3 Total Loss

The total loss is a weighted combination of content and style losses:

$L_total = \alpha\ L_content + \beta\ L_style$

Where:

- α = content weight
- β = style weight

These weights control the balance between structural preservation and artistic stylization.

6. Optimization Process

The output image is initialized as a copy of the content image.

An Adam optimizer is used to iteratively update the pixel values of the output image.

During each iteration:

1. Feature maps are computed for the output image.
2. Content loss and style loss are calculated.
3. Total loss is computed.
4. Gradients are calculated using backpropagation.
5. The output image is updated to minimize total loss.

This process continues for a fixed number of steps until the stylized image converges to a visually acceptable result.

7. Color Preservation Technique

To reduce excessive color transfer from the style image, a color preservation technique is applied using YCbCr color space conversion.

The luminance (Y) component from the stylized output is combined with the chrominance (Cb and Cr) components of the original content image. This ensures that texture is transferred while preserving original color characteristics.

8. Multi-Style Support

The system allows the user to select from multiple predefined artistic styles. The selected style image is used for feature extraction and Gram matrix computation, enabling generation of different artistic outputs without modifying the core architecture.

System Requirements

This section describes the hardware and software requirements necessary to successfully execute the Neural Style Transfer system.

1. Hardware Requirements

The project requires the following minimum hardware configuration:

Minimum Requirements:

- Processor: Intel Core i5 (or equivalent)
- RAM: 8 GB
- Storage: 5 GB free disk space
- GPU: Optional (CPU execution supported)

Recommended Requirements:

- Processor: Intel Core i7 or higher
- RAM: 16 GB
- GPU: NVIDIA GPU with CUDA support (e.g., RTX 3060/4060)
- VRAM: 4 GB or higher
  The use of a CUDA-enabled GPU significantly reduces optimization time and improves performance during iterative training.

2. Software Requirements

The following software components are required:

- Operating System: Windows 10/11 / Linux / macOS
- Python Version: Python 3.8 or higher
- PyTorch (with CUDA support for GPU acceleration)
- Torchvision
- PIL (Python Imaging Library)
- NumPy
- IDE: VS Code / PyCharm / Jupyter Notebook

3. Library Dependencies

The project uses the following major libraries:

- torch → Deep learning framework
- torchvision → Pretrained VGG19 model and transforms
- torch.optim → Optimization algorithms (Adam)
- PIL → Image loading and saving
- numpy → Numerical computations

4. Computational Requirements

Since the model performs optimization-based style transfer:

- Each image requires multiple optimization steps (e.g., 800 iterations).
- Execution time depends on:
    - Image resolution
    - Number of optimization steps
    - Availability of GPU

Typical execution time:

- CPU: 2–5 minutes per image
- GPU: 30–60 seconds per image

System Architecture

The proposed system follows an optimization-based Neural Style Transfer architecture that combines the structural content of one image with the artistic style of another image using a pretrained Convolutional Neural Network. The system consists of several components including input image processing, feature extraction using VGG19, style representation through Gram matrices, loss computation, and iterative optimization to generate the final stylized output image.

1. Input Images

The system requires two types of input images:

1. **Content Image**
   The content image provides the structural layout of the final output image. It contains the objects, shapes, and spatial arrangement that should be preserved.
2. **Style Image**
   The style image provides artistic characteristics such as textures, brush strokes, color patterns, and artistic design elements.

Both images are resized and converted into tensors before being passed into the neural network.

2. Pretrained CNN (VGG19)

A pretrained VGG19 Convolutional Neural Network is used as a **feature extractor**. The network has been previously trained on the ImageNet dataset and is capable of capturing hierarchical visual features from images.

The fully connected classification layers of the network are removed, and only the convolutional layers are used. These layers extract feature maps that represent different levels of visual information.

Lower layers capture edges and simple textures, while deeper layers capture complex structures and object representations.

3. Feature Extraction

Both the content image and the style image are passed through the VGG19 network to extract feature maps from selected convolutional layers.

Content features are extracted from deeper layers to preserve the structure of the content image. Style features are extracted from multiple layers to capture texture patterns and artistic characteristics.

These feature maps serve as representations of the images in feature space.

4. Style Representation using Gram Matrix

To represent artistic style, Gram matrices are computed from the feature maps of the style image. The Gram matrix measures the correlations between feature channels and captures texture patterns independent of spatial location.

By comparing Gram matrices of the output image with those of the style image, the system ensures that the artistic texture is transferred correctly.

5. Loss Computation

Two types of losses are computed during the optimization process:

1. **Content Loss**
   Content loss measures the difference between feature representations of the output image and the content image. It ensures that the structural layout of the content image is preserved.
2. **Style Loss**
   Style loss measures the difference between Gram matrices of the output image and the style image. It ensures that artistic patterns and textures are transferred.

These losses are combined into a total loss function using weighted parameters.

6. Optimization Process

The output image is initialized as a copy of the content image. During each iteration, the output image is passed through the CNN to compute content and style losses.

An optimization algorithm (Adam optimizer) is used to update the pixel values of the output image in order to minimize the total loss function.

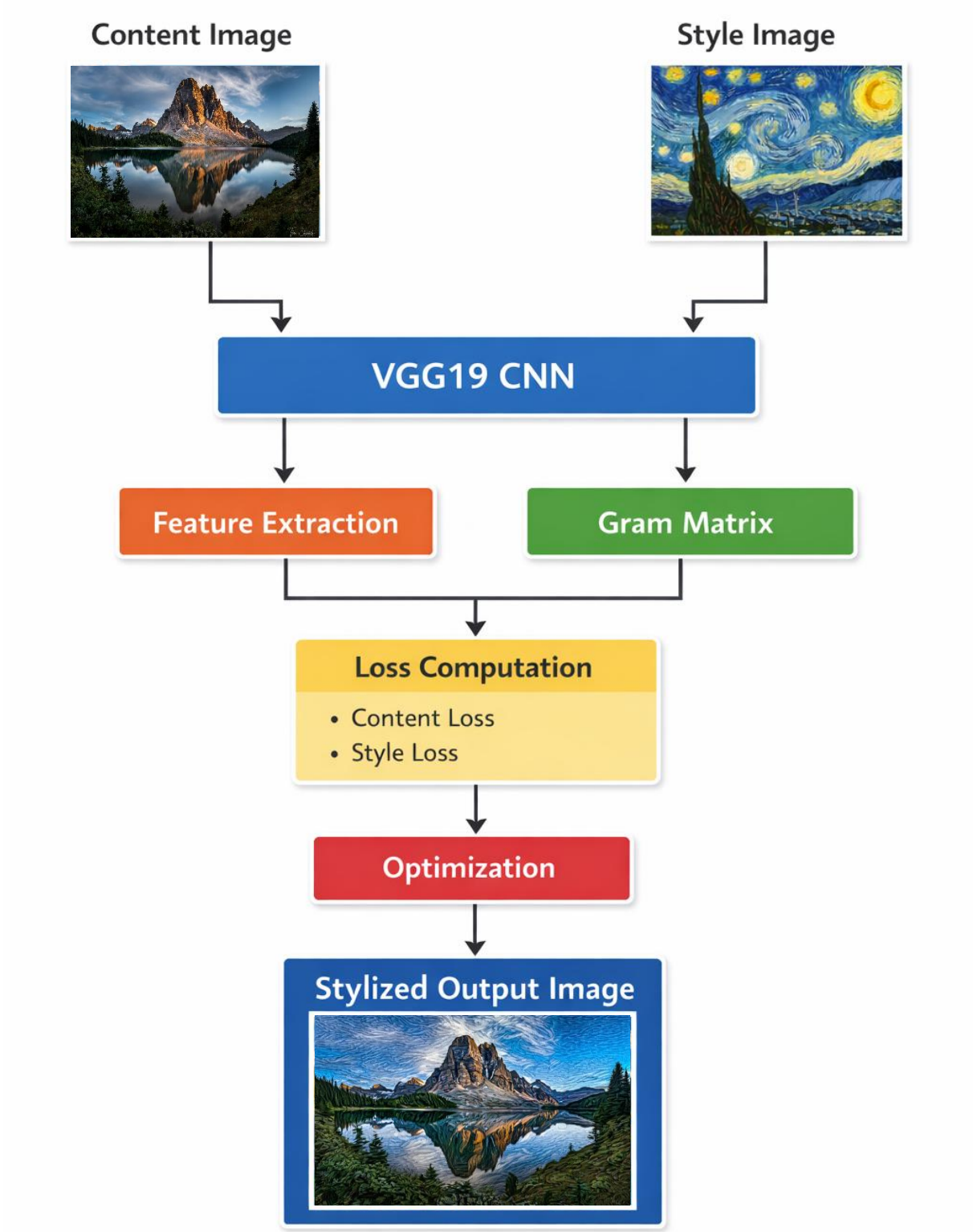
This process continues for a fixed number of iterations until the output image achieves a balance between content structure and artistic style.

7. Stylized Output Image

After optimization converges, the final stylized image is generated. The output preserves the structural elements of the content image while applying the artistic patterns of the selected style image.

The system supports multiple artistic styles, allowing users to generate different stylized outputs by selecting different style images.

Flowchart


Content Image
Style Image
VGG19 CNN
Feature Extraction
Gram Matrix
Loss Computation
• Content Loss
• Style Loss
Optimization
Stylized Output Image

Implementation Details

The Neural Style Transfer system was implemented using Python and deep learning libraries. The implementation uses a pretrained Convolutional Neural Network (VGG19) to extract visual features from images and apply artistic styles to a given content image.

The overall implementation involves loading the images, extracting features using the CNN model, computing style and content losses, and optimizing the output image iteratively until the desired stylized result is obtained.

1. Programming Language

The project was implemented using the **Python programming language** due to its extensive support for deep learning, image processing, and scientific computing libraries.

Python provides flexible tools for implementing neural networks and performing numerical computations efficiently.

2. Libraries Used

Several important libraries were used in the implementation:

PyTorch
PyTorch is the main deep learning framework used for this project. It provides tensor operations, automatic differentiation, and optimization tools required for neural network computation.

Torchvision
Torchvision provides pretrained deep learning models such as VGG19 and also includes image transformation utilities used for resizing and converting images to tensors.

PIL (Python Imaging Library)
PIL is used for loading, processing, and saving images during the execution of the program.

NumPy
NumPy is used for numerical computations and handling multidimensional arrays.

3. Pretrained Model (VGG19)

The implementation uses the **VGG19 Convolutional Neural Network**, which is pretrained on the ImageNet dataset. Only the convolutional layers of the network are used to extract image features.

The fully connected classification layers are not required for style transfer, so only the feature extraction part of the network is utilized.

The pretrained weights of the network remain fixed during execution, meaning the network itself is not trained further. Instead, the output image is optimized to minimize the defined loss functions.

4. Image Processing

The input images are first preprocessed before being passed into the neural network.

The preprocessing steps include:

- Resizing the images to a fixed size (512 × 512 pixels)
- Converting the images into tensor format
- Normalizing the images for processing within the neural network

These steps ensure that the images can be processed correctly by the CNN model.

5. Optimization Process

The output image is initialized as a clone of the content image and is iteratively updated during the optimization process.

The **Adam optimizer** is used to minimize the total loss function, which is a weighted combination of content loss and style loss.

The optimization process runs for a fixed number of iterations, gradually updating the output image so that it preserves the structure of the content image while adopting the artistic style of the style image.

6. Parameter Settings

The following parameters were used during implementation:

Number of optimization steps: **800 iterations**

Content weight: **15**

Style weight: **100**

These parameters control the balance between preserving the original image structure and transferring the artistic style.

7. Color Preservation Technique

To prevent excessive color distortion caused by style transfer, a color preservation method was implemented using the **YCbCr color space**.

The luminance component from the stylized image is combined with the chrominance components of the original content image. This allows the output image to retain the original colors while still applying the artistic texture from the style image.

Results and Output Analysis

This section presents the results obtained from the Neural Style Transfer model. The system was tested using different types of content images such as human portraits, animals, and natural landscapes. Various artistic styles were applied including Starry Night, Watercolor, Oil Painting, Abstract Art, Swirl Art, and Color Pencil Art.

Figure: Execution of neural style transfer program showing style selection and output generation

```
C:\Users\zara\.vscode\python by Z\ImageToArtProject>py -3.11 style_transfer.py
Using device: cuda

Choose Style:
1 - Starry Night
2 - Watercolor Painting
3 - Oil Painting
4 - Abstract Art
5 - Psychedelic Swirl Art
6 - Color Pencil Art
Enter choice (1-6): 6

Applying style...

Step: 0
Step: 50
Step: 100
Step: 150
Step: 200
Step: 250
Step: 300
Step: 350
Step: 400
Step: 450
Step: 500
Step: 550
Step: 600
Step: 650
Step: 700
Step: 750

DONE. output.jpg saved.
```

The objective of the experiment was to evaluate how effectively the model transfers the artistic style while preserving the structural content of the original image.

The results demonstrate that the implemented model successfully combines the content and style representations to generate visually appealing stylized images.

Result 1 – Starry Night Style Transfer

Content Image

Style Image

Stylized Output

The Starry Night style, inspired by Vincent van Gogh's famous painting, introduces swirling brush strokes and textured patterns into the content image. The model successfully preserves the main structure of the original image while applying the characteristic brush stroke texture and artistic patterns of the Starry Night style.

The output image demonstrates how the neural network captures the style features such as swirling patterns and applies them to different regions of the content image.

Result 2 – Watercolor Style Transfer

Content Image

Style Image

Stylized Output

In the watercolor style transfer, the output image displays soft color blending and smooth brush textures. The model applies watercolor-like patterns to the content image while maintaining the general structure and shape of the objects.

The resulting image has a smoother and more artistic appearance compared to the original photograph.

Result 3 – Oil Painting Style Transfer

Content Image

Style Image

Stylized Output

The oil painting style introduces thicker brush strokes and stronger color contrasts into the content image. The output image shows how the neural network applies rich textures and artistic patterns characteristic of oil paintings.

The model successfully transfers the artistic texture while preserving the overall layout of the original image.

Result 4 – Abstract Art Style Transfer

Content Image

Style Image

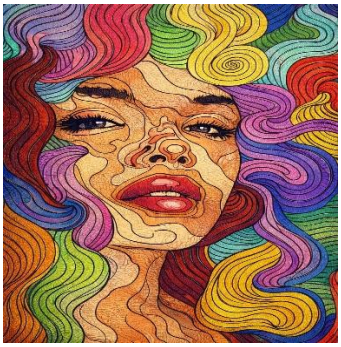
Stylized Output

The abstract style transforms the content image by introducing geometric patterns and vibrant color variations. The neural network captures the complex artistic patterns of the abstract painting and applies them to the structure of the original image.

The output image demonstrates a creative transformation where the original structure is still recognizable but represented in a more artistic form.

Result 5 – Psychedelic Swirl Art

Content Image



Style Image

Stylized Output

The swirl art style introduces dynamic flowing patterns and colorful curved structures into the content image. The output image displays a combination of the original object structure with the swirling artistic elements extracted from the style image.

This style produces a visually vibrant and artistic transformation of the content image

Result 6 – Color Pencil Style

Content Image

Style Image

Stylized Output

The color pencil style gives the output image a hand-drawn artistic appearance. The neural network captures the texture and shading patterns typical of colored pencil drawings and applies them to the content image.

The final output resembles a sketch-like illustration while preserving the structure and details of the original image.

The experimental results demonstrate that the neural style transfer model successfully combines the content and style representations to generate artistic images. The model effectively preserves the structure of the content image while transferring the texture and patterns of the selected artistic style.

Different styles produce unique visual effects, highlighting the flexibility and creative potential of neural style transfer techniques.

Applications

Neural Style Transfer has several practical applications in the fields of digital art, entertainment, and image processing. By combining the content of one image with the artistic style of another, this technique allows users to generate creative and visually appealing images.

One of the major applications of Neural Style Transfer is in **digital art creation**. Artists and designers can use this technique to transform ordinary photographs into artistic paintings inspired by famous artists such as Van Gogh or Picasso. This helps artists experiment with different artistic styles without manually recreating them.

Another important application is in the **film and animation industry**. Neural Style Transfer can be used to apply consistent artistic styles to video frames, creating unique visual effects and stylized animations. This technology can enhance the visual storytelling process in movies and animated content.

Neural Style Transfer is also widely used in **photo editing applications and mobile apps**. Many modern photo editing tools allow users to apply artistic filters to their photographs using style transfer techniques. This enables users to convert their photos into watercolor paintings, oil paintings, sketches, and other artistic styles with just a few clicks.

In addition, the technique has applications in **social media and digital content creation**. Content creators often use stylized images to produce visually attractive posts and graphics. Neural Style Transfer helps generate unique visual content that stands out on digital platforms.

Furthermore, this technology can be applied in **education and creative design tools**. Students and designers can use style transfer models to explore artistic creativity and understand how different artistic styles influence visual appearance.

Overall, Neural Style Transfer provides powerful tools for artistic expression and creative image transformation across various digital platforms.

Advantages

Neural Style Transfer provides several advantages in the field of digital image processing and artistic image generation. One of the primary advantages is its ability to **combine artistic style with real-world images automatically**. The technique allows ordinary photographs to be transformed into artistic images without requiring manual drawing or editing skills.

Another major advantage is **high-quality artistic output**. By using deep learning models such as Convolutional Neural Networks, the system can extract detailed textures and patterns from style images and apply them to content images while preserving the overall structure of the original image.

Neural Style Transfer is also **highly flexible and customizable**. Different artistic styles can be applied simply by changing the style image. This allows users to experiment with a wide range of artistic transformations such as watercolor, oil painting, abstract art, and sketch styles.

The technique also **reduces manual effort in digital art creation**. Designers and content creators can generate artistic images quickly without spending significant time on manual painting or graphic editing.

Another advantage is that **pretrained models can be used without training from scratch**. In this project, the VGG19 model trained on the ImageNet dataset was used to extract image features, which significantly reduces the computational resources and time required for training a new model.

Additionally, Neural Style Transfer has **broad applicability across multiple domains**, including digital media, entertainment, advertising, photography, and graphic design.

Overall, Neural Style Transfer provides an efficient and powerful approach for generating creative visual content using deep learning techniques.

Limitations

Despite its ability to generate visually appealing artistic images, Neural Style Transfer has several limitations. One of the primary limitations is the **high computational cost** of the optimization-based approach. The model requires multiple iterations to optimize the output image, which can take a significant amount of time, especially when running on a CPU without GPU acceleration.

Another limitation is that the **process is not real-time**. Since the system performs iterative optimization for each image, generating stylized outputs can take several seconds or minutes depending on the number of iterations and the hardware used.

Neural Style Transfer may also sometimes cause **distortion of the original image structure**. If the style weight is too high, the artistic patterns may overpower the content features, resulting in images where the original structure becomes less recognizable.

Another challenge is **limited control over artistic details**. Although the user can adjust content and style weights, it is difficult to precisely control how specific textures or patterns appear in different parts of the image.

In addition, the quality of the final output is **highly dependent on the chosen style image**. Some styles may transfer well to the content image, while others may produce visually cluttered or unnatural results.

Finally, the optimization-based approach used in this project is **computationally intensive compared to feed-forward style transfer networks**, which can generate stylized images much faster.

Future Scope

Although the proposed Neural Style Transfer system successfully generates artistic images by combining content and style features, several improvements can be explored in future work.

One possible enhancement is the implementation of **real-time style transfer using feed-forward neural networks**. Unlike optimization-based methods, feed-forward models can generate stylized images instantly, making them suitable for real-time applications such as mobile apps and video processing.

Another potential improvement is the extension of the system to **video style transfer**. By applying the style transfer technique to consecutive video frames while maintaining temporal consistency, it would be possible to generate stylized animations and artistic video effects.

The system could also be improved by providing **greater user control over stylization intensity**. Future versions of the project may include adjustable parameters or graphical interfaces that allow users to control the strength of the applied style.

Another interesting direction is the use of **Generative Adversarial Networks (GANs)** for artistic image generation. GAN-based approaches can produce more diverse and realistic artistic transformations compared to traditional optimization-based methods.

Additionally, future systems could support **a larger library of artistic styles** and allow users to upload custom style images to generate personalized artistic outputs.

Overall, further research and improvements in deep learning techniques can make Neural Style Transfer faster, more controllable, and more suitable for real-world creative applications.

Conclusion

In this project, an optimization-based Neural Style Transfer system was successfully implemented using deep learning techniques. The system combines the structural content of one image with the artistic style of another image to generate visually appealing stylized outputs. A pretrained VGG19 Convolutional Neural Network was used as a feature extractor to obtain hierarchical image representations.

The implementation utilized feature extraction from selected convolutional layers, Gram matrices to represent artistic style, and loss functions to preserve both content and style characteristics. By minimizing a weighted combination of content loss and style loss using the Adam optimization algorithm, the output image was iteratively refined until a satisfactory stylized result was achieved.

The system supports multiple artistic styles including Starry Night, Watercolor, Oil Painting, Abstract Art, Psychedelic Swirl Art, and Color Pencil Art. Experimental results demonstrated that the model effectively preserves the structure of the content image while transferring artistic textures and patterns from the selected style image.

Overall, this project demonstrates how pretrained deep learning models and optimization techniques can be applied to create artistic image transformations. The work highlights the potential of neural networks in creative applications and showcases the integration of machine learning concepts such as convolutional neural networks, feature extraction, Gram matrix representation, and loss-based optimization.

References

1. Gatys, L. A., Ecker, A. S., & Bethge, M. (2015). *A Neural Algorithm of Artistic Style*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). https://arxiv.org/abs/1508.06576
2. Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG19)*. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/1409.1556
3. Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems (NeurIPS).
4. PyTorch Documentation. *PyTorch Official Documentation*. https://pytorch.org/docs/stable/index.html
5. Torchvision Documentation. *Torchvision Models and Image Transformations*. https://pytorch.org/vision/stable/index.html
6. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.